

滴滴国际化金融二面面经复盘

一、面试概况

- 面试时间：2026-09-04 17:36
- 录音总时长：约 40 分 39 秒
- 正式面试时长：约 36 分 32 秒（02:36 开始，39:08 结束）
- 面试方向：滴滴国际化金融服务端开发二面
- 主要内容：自我介绍、国际支付风控业务、客诉机审项目、链路性能与缓存一致性、工作流编排、Agent 与 AI 编程、知识库与 Skill 治理、英文项目介绍、候选人反问
- 现场编码：无。面试官说明本轮更关注风控业务理解，不代表其他面试官不会考算法

二、整理说明

本文根据 [滴滴二面_原文.docx](#) 的语音转写整理。原文只有转写文本，没有表格、图片或共享屏幕题面。开场前的设备测试、自言自语和正式结束后的环境录音不纳入问题清单。

原文存在较多同音字和英文识别错误，本文结合上下文将部分术语统一为 [Risk Platform](#)、[approve](#)、[reject](#)、[review](#)、Redis、[SET NX](#)、Trace ID、Workflow、PRD、Code Review、Skill 等。以下内部名称仍无法仅凭转写确认，面试前应以真实项目文档为准：

1. 配置平台被转写为 [TCC](#) 或 [PCC](#)。
2. 表达式解析能力被转写为 [Go Values](#)。
3. 承载业务链路的服务名被转写为 [Conan NRT](#) 或近似读音。
4. AI 编程工具、内部文档工具和模型名称的转写不稳定，不能直接照搬为正式产品名。

需要特别保持三类边界：

1. 本人负责的是运营平台中的具体客诉机审业务链路，包括字段接入、规则与动作编排、下游调用、异常兜底和联调；底层风控模型、通用配置平台、表达式引擎及处罚中心不是本人从零开发。
2. 运营、产品和风控策略团队负责业务审核口径与风险阈值，研发负责把已确认规则转换成安全、可执行、可观测的系统能力。不能把业务规则说成个人拍脑袋决定。
3. 原文没有给出真实 QPS、P99、自动化率、准确率或节省人力数据。下文只给出可验证的指标框架，不补造线上结果。

三、本场复盘

这轮面试表面上围绕项目和 AI 工具展开，实际重点考察三件事：能否从上下游讲清风控业务、能否准确界定个人贡献、能否把工程方案与业务影响联系起来。

现场相对可取的地方：

1. 能够说明客诉机审链路位于处罚后的运营阶段，并主动澄清处罚中心只是被调用方。
2. 性能问题知道先通过案件 ID、Trace ID 和链路日志定位具体慢点，而不是直接改代码。
3. 缓存更新顺序能想到“数据库提交后删除缓存”的 [Cache Aside](#) 基线。
4. 工作流优化意识到只有相互独立的节点才适合并行。
5. AI 编程流程覆盖了需求澄清、方案设计、分模块实现、人工评审、测试和 QA，而不是把整项需求一次性交给模型。

主要问题：

1. **对风控业务的回答包含较多猜测。** 不清楚底层引擎时，应明确“我没有参与该模块”，再给出通用架构理解，不能把验证中心、处罚中心或数据库查询路径说成真实实现。
2. **没有先区分事前、事中和事后。** 本人项目是处罚后的申诉/客诉自动审核，不等于在线交易风控决策引擎，也不应默认所有案件都是盗刷或退款。
3. **缓存一致性补救方案不准确。** 给所有读写请求加 Redis 分布式锁成本高，也不能自动解决数据库、缓存和线程暂停之间的一致性。应回答数据库事实源、提交后删缓存、删除可靠重试、缓存重建防并发和版本控制。
4. **热点识别方案过于抽象。** “维护一个数据结构”不够，需要说明采集什么指标、怎样控制高基数开销、如何识别持续热点与突发热点，以及热点消退后的退出策略。
5. **链路优化没有形成证据闭环。** 不能一看到串行 RPC 就改并行；应先看 Trace 和关键路径，再分析数据依赖、顺序约束、副作用、下游容量和失败语义，最后灰度验证主链路与下游指标。
6. **AI 和 Skill 回答术语很多，但治理不足。** 需要补充知识分层、检索路由、权限、版本、Owner、评测、去重和废弃机制，不能只依赖一份很长的说明文件。
7. **英文介绍临场组织失败。** 应提前准备稳定的 5 句版本，使用简单句讲清团队、项目、职责、结果边界和收获，不追求复杂词汇。

四、所有问题清单

A. 开场与风控业务

1. 请做一个简单的自我介绍。
2. 一个完整的风控系统通常包含哪些模块，分别承担什么功能？
3. 你所在链路处理的是客诉、可疑交易，还是其他类型的风险？
4. 风控决策引擎通常通过哪些步骤和技术识别一笔事件是否有风险？
5. 风控规则一般怎样实现？面对海量风控事件，系统怎样保证处理能力？
6. 决策引擎怎样获知商户或用户的历史黑灰产记录？哪些系统会协作完成这件事？

B. 客诉机审项目与个人边界

7. 你在整个风控体系中具体负责哪一部分？
8. 请展开介绍你负责的风控客诉机审链路。
9. 为什么原来由人工审核的案件可以转为机审？你们完成了哪些步骤？
10. 机审规则和配置来自哪里？产品、运营和研发分别负责什么？
11. 除了在配置平台填写规则，你还完成了哪些开发工作？
12. 这条后台审核服务对时间敏感吗？应该怎样定义它的时延要求？

C. 性能、缓存与 workflow 编排

13. 一笔案件原来 1 秒处理完，现在突然需要 20 秒，可能是什么原因？你会怎样定位和解决？
14. 除了下游 RPC 超时，还有哪些因素会让整条链路变慢？
15. 怎样判断哪些数据或 Key 是热点？怎样识别静态热点和突发热点？
16. 动态热点进入 Redis 后，数据库与缓存的更新顺序应该怎样设计？
17. 更新数据库期间仍有并发读请求时，怎样处理缓存与数据库短暂不一致的问题？
18. 处罚中心也是你负责开发的吗？你与它是什么关系？
19. 你的工作是否只是把已有能力串起来？个人真正负责的技术内容是什么？
20. 对决策 workflow 做优化时，有哪些可以复用的实践？

21. workflows 配置是完全手工完成的吗？怎样保证配置正确并安全发布？
22. 怎样判断节点应该串行还是并行？如何在保证正确性的前提下压缩关键路径？

D. Agent、AI 编程与知识治理

23. 实习期间是否接触过 Agent 和 AI 编程？分别用在什么场景？
24. 从拿到 PRD 到代码上线，你会怎样与 Codex 等 AI 编程工具协作？
25. 需求澄清阶段，你怎样判断应该向产品、运营或研发同学询问哪些问题？
26. 团队怎样避免实习生或新人反复询问同一批项目背景问题？
27. 大量项目文档应该怎样组织，才能让新人快速建立全局认识，而不是逐篇阅读？
28. 面对十几个微服务和大量文档，怎样让上下文有限的 AI 快速、低成本地找到相关信息？
29. AI 写代码时喜欢新造能力、不复用系统已有实现，应该怎样引导和约束？
30. 当规则很多、长指令容易被忽略时，怎样让 AI 在开发的具体阶段知道应复用哪些能力？
31. 团队 Skill 越来越多时，怎样做分类、路由、版本和生命周期管理？

E. 英文表达

32. 请用英语简单介绍一个实习项目，包括所在团队、项目目标、个人贡献和收获。

F. 候选人反问与面试反馈

33. 为什么本轮没有安排算法题？
34. 面试官怎样评价本场表现？后续最需要改进什么？
35. 滴滴国际化金融团队目前怎样使用 AI 编程，AI 生成代码的参与程度如何？

五、逐题参考回答

A. 开场与风控业务

1. 请做一个简单的自我介绍

建议时长：60~90 秒。

面试官您好，我是吉林大学计算机相关专业的 2027 届本科生，求职方向是服务端开发。我有两段后端实习经历：今年 1 月到 3 月在途游游戏中台参与客服系统、玩家召回和支付风控相关需求；4 月到 8 月在字节国际支付风控合规团队参与运营平台的后端开发。

在字节期间，我投入较多的是风控客诉自动审核链路。原来部分申诉案件主要由人工处理，我们把上游案件字段、内部服务结果和已有 Agent 的结构化信号汇聚起来，再通过可配置规则完成自动判断；证据不足、结果冲突或依赖异常的案件继续转人工。我主要负责字段接入、规则和动作编排、下游调用、异常兜底及联调，不负责底层模型训练或风控阈值制定。

这两段经历让我对 Go/Java 后端、MySQL、Redis、跨服务调用和分布式链路有了更具体的认识。我希望继续做复杂业务场景下的服务端开发，进一步提升系统设计、稳定性和业务抽象能力。

2. 一个完整的风控系统通常包含哪些模块，分别承担什么功能

建议时长：75~90 秒。

我会先按一笔风险事件的生命周期来讲。入口层负责接入交易、登录、支付或申诉事件，并完成鉴权、Schema 校验、限流和去重；数据与特征层把用户、设备、商户、账户、历史行为和名单等信息整理成可供决策使用的

结构化事实；决策层通过规则、模型或二者组合输出通过、拒绝、挑战验证或转人工等结果。

结果随后进入处置层，执行拦截、限制、二次验证、通知等动作，并把完整决策依据写入审计记录。对于无法自动判断或已经产生争议的案件，还需要案件中心、申诉/客诉平台和人工审核。最外层还要有配置发布、版本、灰度、监控、回滚、权限和合规审计。

这只是通用拆分。就我实际参与的范围而言，我在处罚后的运营/申诉阶段做具体机审链路，没有参与在线风控模型、底层特征平台或整个风控体系的建设。

3. 你所在链路处理的是客诉、可疑交易，还是其他类型的风险

建议时长：45~60 秒。

更准确地说，我负责的是**处罚后的申诉或客诉自动审核链路**，属于事后运营阶段，而不是在线交易发生时的实时决策引擎。

上游风控系统已经根据自身策略对某类支付主体或商户案件作出识别和处置，之后一部分主体会发起申诉，案件流转到运营平台。我们针对边界明确的案件尝试自动审核，不能确定的仍然转 BPO 人工处理。

录音里提到了可疑支付、黑灰产和盗刷，但我没有参与底层规则制定，也不能确认每个案件的具体风险类型。因此正式面试时只讲自己能核实的案件范围、输入字段和处理流程，不猜测真实阈值或风险标签。

4. 风控决策引擎通常通过哪些步骤和技术识别一笔事件是否有风险

建议时长：75~90 秒。

通用流程可以概括为“事件接入、事实补全、风险判断、执行处置、结果反馈”。事件到达后，系统先校验身份、请求完整性和幂等键，再根据用户、商户、设备、账户等实体获取实时特征、历史行为、名单和关系信息。然后由确定性规则、统计模型或机器学习模型综合评分，输出通过、拒绝、增强验证或人工审核。

高风险系统不能只返回一个黑盒结果，还需要记录命中的规则版本、关键特征、模型版本和决策原因，支持审计、申诉和回溯。处置结果以及后续人工结论还会反馈给规则和模型团队，用于发现漂移和调整策略。

我会强调这是我对通用架构的理解。实际公司内部使用哪些特征、模型和阈值，只有负责对应模块的团队才能确认，我不会把这部分说成自己的实现。

5. 风控规则一般怎样实现？面对海量风控事件，系统怎样保证处理能力

建议时长：90 秒。

规则通常不是把任意代码直接放进配置，而是由平台提供受限 DSL 或表达式能力。规则配置包含适用范围、字段、操作符、阈值、优先级、版本和动作；发布前需要做 Schema 校验、类型校验、冲突检测、审批和灰度，运行时只能访问白名单字段并选择已经注册的动作。

处理海量事件时，首先让在线决策服务尽量无状态化并水平扩展；规则和只读基础数据可以按版本下发到本地内存或高可用缓存，避免每次决策都远程查库。历史聚合特征可以通过流式计算提前生成，由在线特征服务提供低延迟读取。系统还要设置分层超时、有界线程池、背压、限流和降级，避免一个慢下游拖垮整条链路。

最后要用 QPS、P95/P99、超时率、降级率、规则版本和下游容量做观测。多机房和负载均衡只是承载流量的一部分，不能替代规则缓存、特征预计算和容量治理。

6. 决策引擎怎样获知商户或用户的历史黑灰产记录？哪些系统会协作完成这件事

建议时长：75~90 秒。

不建议让决策引擎临时连接多个业务库，直接拼出所有历史。更常见的做法是由用户画像、风险档案、名单、设备关系、处罚记录或在线特征服务分别维护自己负责的事实，再通过稳定接口或预计算特征提供给决策层。

对低频、强一致的信息，可以按业务主键调用权威服务；对高频特征，可以由离线和流式链路预聚合到在线特征存储，并带上时间窗口、数据版本和过期时间。决策引擎只消费经过权限控制的结构化结果，例如“近 30 天风险事件次数”或“是否命中有效名单”，而不是自己理解其他系统的底层表。

我实际没有参与这条底层特征链路，所以正式回答应该说“通用设计上会这样做”，不能断言公司内部一定查询验证中心、处罚中心的某张表。

B. 客诉机审项目与个人边界

7. 你在整个风控体系中具体负责哪一部分

建议时长：45~60 秒。

我负责的是运营平台中的一条风控客诉自动审核业务链路。上游把案件和结构化字段传入后，我所在服务补充必要的内部能力结果，根据已有配置规则作出 `approve`、`reject` 或 `manual review` 判断，再编排并调用允许的下游动作。

我的工作包括字段接入、配置结构、规则上下文、动作注册与编排、RPC 调用、异常兜底、测试和联调。通用配置平台、表达式引擎、底层 Agent、风控阈值和处罚中心由其他团队负责，我只是接入和使用这些能力。

8. 请展开介绍你负责的风控客诉机审链路

建议时长：90 秒。

项目背景是国际支付的部分风控申诉案件原先主要依赖 BPO 人工审核，处理成本较高，而且一些边界明确的案件具有重复、可规则化的特点。我们的目标不是把所有案件都自动化，而是先让满足清晰条件的一部分案件进入机审，其余继续人工处理。

链路收到案件后，先校验案件类型、状态和必要字段，再加载上游标签、历史状态以及内部 Agent 或其他服务提供的结构化结果。之后把这些事实组装成规则上下文，交给已有表达式能力求值。规则明确命中时输出通过或拒绝，并调用配置允许的业务动作；字段缺失、结果冲突、调用超时或规则未覆盖时转人工。

工程上还要保证请求幂等、动作白名单、参数校验、规则版本、超时重试、审计日志和异常告警。我的贡献集中在这条具体业务链路，不负责公司已有平台和算法能力包装成个人从零搭建。

9. 为什么原来由人工审核的案件可以转为机审？你们完成了哪些步骤

建议时长：75~90 秒。

不是因为“AI 可以替代人工”，而是其中一部分案件的输入字段稳定、判断标准明确、结果动作可审计，而且存在安全的人工兜底。只有同时满足这些条件，才适合从人工迁移到自动化。

落地时我会分五步说明：第一，和运营、产品梳理人工审核记录，确定目标案件范围和不可自动化的风险边界；第二，把判断所需的材料、字段和内部能力结果结构化；第三，把规则和动作做成受控配置，代码侧维护白名单和校验；第四，先做离线回放或影子判断，与人工结果对比，再按业务线或区域灰度；第五，持续观察覆盖率、人工抽检一致率、误放/误拒、依赖失败和转人工原因，异常时可以快速回滚。

原文没有给出真实灰度方式和质量数据，因此这些是规范化回答框架。没有实际参与的步骤要用“方案应当”表述。

10. 机审规则和配置来自哪里？产品、运营和研发分别负责什么

建议时长：60 秒。

业务规则不是研发个人决定的。运营最了解人工审核过程和历史案例，会沉淀案件口径、判断条件和例外；产品负责明确目标范围、流程、成功指标和跨团队需求；风控或策略同学对风险阈值和合规边界负责。

研发的职责是把这些业务口径转成可执行系统：确认字段来源与类型，定义配置 Schema，接入规则上下文和动作执行器，补充校验、版本、权限、灰度、回滚、监控和审计。如果业务规则存在歧义，研发要通过样例和边界用例推动确认，而不是自己猜一个阈值上线。

11. 除了在配置平台填写规则，你还完成了哪些开发工作

建议时长：60~75 秒。

配置只是链路的一部分。业务服务还需要接收和校验上游案件，完成字段映射，调用内部服务获取补充信息，把结果组装成规则上下文，并把规则结果映射到受控动作。

我还需要处理 RPC 超时、部分失败和重复请求，给动作设计幂等键或状态条件，保证不确定时转人工而不是误判；同时补充日志、Trace、指标、单元测试、联调和发布验证。对于新的字段来源或全新动作，仍然必须开发代码和接口，配置化不等于任何变化都能零代码上线。

12. 这条后台审核服务对时间敏感吗？应该怎样定义它的时延要求

建议时长：60 秒。

它虽然不是支付授权主链路，但仍然有明确时延要求。上游通常有调用超时，运营侧也关心案件处理时效；如果内部 Agent、规则或下游动作过慢，会导致整体超时、重复请求或转人工比例上升。

我会先以业务 SLA 为起点，再把总预算分配给排队、数据加载、规则计算和动作执行，并分别监控 P50、P95、P99、超时率和 fallback 比例。不同结果可以采用不同策略：必须同步返回的判断走短超时；非关键通知可以异步；超时或结果不确定时转人工，不能为了低延迟强行给出高风险结论。

实际阈值必须以项目监控和上游约定为准，原文没有提供数字。

C. 性能、缓存与 workflow 编排

13. 一笔案件原来 1 秒处理完，现在突然需要 20 秒，可能是什么原因？你会怎样定位和解决

建议时长：90 秒。

我不会先猜某个 RPC，也不会直接把串行改并行。第一步用案件 ID 或请求 ID 找到 Trace，把 20 秒拆成排队、服务内部计算、数据库/缓存访问、各个 RPC、重试和日志等 Span，先确认时间花在谁身上；再比较同版本的正常请求，判断是单请求异常、某个业务分支，还是全局容量问题。

常见原因包括下游变慢或重试叠加、连接池或线程池排队、慢 SQL、缓存穿透、锁竞争、GC 暂停、CPU/网络抖动、配置规则复杂度上升和请求体异常变大。定位后再按原因处理：慢 SQL 看执行计划和索引；线程池看队列、拒绝和依赖容量；RPC 设置合理超时、有限重试、熔断和降级；独立读节点可做有界并行；稳定数据才考虑缓存或批量查询。

最后通过灰度对比整体 P95/P99、错误率、fallback 比例以及下游 QPS、超时和资源使用，防止主链路变快却把下游压垮。

14. 除了下游 RPC 超时，还有哪些因素会让整条链路变慢

建议时长：60 秒。

可以按资源和阶段分类。入口可能在线程池、连接池或消息队列排队；服务内部可能有锁竞争、死循环、低效序列化、大对象分配或 GC 暂停；数据层可能有慢 SQL、索引失效、热点行、连接池耗尽、缓存 miss 或大 Key；网络层可能有 DNS、建连、TLS、丢包和跨地域抖动。

还要关注业务层的隐性放大，例如一个新规则触发更多分支、串行 fan-out、超时后多次重试，或者日志同步写入过多。排查时用 Trace 看关键路径，用指标看资源饱和，用日志确认业务分支，三者要能够用同一个请求 ID 关联起来。

15. 怎样判断哪些数据或 Key 是热点？怎样识别静态热点和突发热点

建议时长：75~90 秒。

先定义热点：它可能是访问 QPS 高、带宽大、延迟高、缓存 miss 多，也可能是写冲突严重，不能只看访问次数。静态热点可以从历史监控、业务日历和容量报表识别，例如固定活动配置或头部商户；突发热点则要依赖分钟级甚至秒级的请求采样和告警。

实现上可以在网关、客户端或缓存代理按 Key/业务 ID 采样计数，对高基数字段使用滑动时间窗、Count-Min Sketch、Space-Saving 或 Top-K 聚合，避免给每个 Key 建永久计数器。还要结合节点 CPU、网络流量、命中率和单 Key 延迟，防止把“大 Value”误判成单纯高频。

检测到热点后再选择本地缓存、Redis、副本、请求合并或限流，并设置 TTL、衰减或退出阈值，让突发热点消退后自动降级，避免热点名单无限增长。

16. 动态热点进入 Redis 后，数据库与缓存的更新顺序应该怎样设计

建议时长：60~75 秒。

通用基线是 Cache Aside：MySQL 作为事实源。读请求先查缓存，未命中再查数据库并回填；写请求先在事务中更新数据库，提交成功后删除缓存，而不是同时直接修改数据库和缓存两个副本。

删除缓存失败时需要可靠重试，可以使用 Outbox、binlog/CDC 加消息队列，或由专门补偿任务再次删除；TTL 只是最后的自愈手段，不能替代可靠处理。回填时应设置合理 TTL 和随机抖动，避免大量 Key 同时失效。

动态热点识别只决定哪些数据值得缓存，不改变事实源和一致性原则。若业务要求强一致，就应绕过缓存读主库，或采用带版本的专门方案，不能宣称普通 Cache Aside 能做到任何时刻都一致。

17. 更新数据库期间仍有并发读请求时，怎样处理缓存与数据库短暂不一致的问题

建议时长：75~90 秒。

先承认 Cache Aside 通常提供最终一致，不保证零窗口。一个典型竞态是：缓存刚好 miss，读线程先读到数据库旧值；写线程随后提交新值并删除缓存；最后读线程又把旧值回填，造成一段时间的脏缓存。

基础措施是数据库作为事实源、提交后删缓存、删除失败可靠重试和较短 TTL。对并发重建，可以按 Key 做请求合并或互斥，让一个线程回源，但不需要给所有正常读写都加分布式锁。更严格时可以让缓存值携带数据版

本，回填前校验版本，或者由 binlog/CDC 事件按单调版本删除/更新缓存，拒绝旧版本覆盖新版本。

延迟双删可以降低部分竞态概率，但依赖延迟估计，不是严格证明。对于余额、风控最终状态等强一致读取，应直接访问权威存储或使用业务版本和条件更新，不能只靠 Redis 锁保证正确性。

18. 处罚中心也是你负责开发的吗？你与它是什么关系

建议时长：30~45 秒。

不是。处罚中心是独立下游能力，由对应团队负责开发和维护。我负责的运营机审链路在规则命中后，根据受控动作调用处罚中心或其他服务，并处理参数校验、超时、错误码、幂等和异常兜底。

因此准确表述是“我接入并编排了处罚能力”，不是“我开发了处罚中心”。

19. 你的工作是否只是把已有能力串起来？个人真正负责的技术内容是什么

建议时长：60~75 秒。

从系统边界看，这项工作确实属于业务编排，但业务编排不等于简单地按顺序调用几个接口。真正需要解决的是把变化的案件字段、规则和动作抽象成稳定主流程，并处理分布式调用中的重复、超时、部分成功和人工兜底。

我负责把上游字段映射到规则上下文，定义配置结构和动作注册方式，完成 RPC 接入、执行顺序、错误处理、日志 Trace、测试和联调。通用平台和下游能力是已有的，我的价值是把它们组合成一条可维护、可扩展、失败时不会误处理案件的业务链路。

正式面试时还要结合真实代码说明一个具体难点和解决过程，避免只停留在“我编排了 Workflow”这一层。

20. 对决策工作流做优化时，有哪些可以复用的实践

建议时长：90 秒。

我会按“先观测、再建模、后优化、最后验证”处理。先用 Trace 找到关键路径和长尾节点，区分耗时来自排队、计算、存储还是外部依赖；再把工作流建模成有向无环图，标出数据依赖、顺序约束、副作用和失败补偿。

独立、只读且下游容量允许的节点可以有界并行；重复获取的数据可以批量查询、请求合并或缓存；非关键通知可以异步；每个节点设置独立超时、有限重试、熔断和 fallback。存在副作用的动作需要幂等键、状态机和必要的补偿，不能仅靠并行提高速度。

上线时先做压测和故障注入，再小流量灰度，比较端到端 P95/P99、成功率、fallback 比例以及每个下游的 QPS、超时和资源利用率。只有主链路与下游都稳定，才算真正优化。

21. 工作流配置是完全手工完成的吗？怎样保证配置正确并安全发布

建议时长：60 秒。

业务口径由运营、产品和策略同学提供，研发把它转换成平台支持的结构化配置，因此它不是某个开发者凭经验随意填写，也不能把未经审核的 AI 输出直接发布。

配置发布前至少要做字段类型和 Schema 校验、动作白名单、参数范围、规则冲突、样例回放和权限审批；发布时带版本，按区域或流量灰度，保留审计记录和一键回滚；运行后观察命中、转人工、动作失败和质量指标。

AI 可以帮助解释文档、生成候选配置或测试用例，但最终配置必须由业务负责人和研发共同核对。本人实际参与了哪些校验与发布环节，需要以项目事实为准。

22. 怎样判断节点应该串行还是并行？如何在保证正确性的前提下压缩关键路径

建议时长：75~90 秒。

判断依据不是“接口看起来互不相同”，而是依赖关系和失败语义。若 B 的输入依赖 A 的输出、二者修改同一业务状态、存在必须保持的顺序，或者失败后需要按顺序补偿，就应串行。若两个节点只读取独立数据、没有共享副作用、结果最后统一汇总，而且下游容量允许，才适合并行。

我会先画依赖 DAG，计算关键路径，把独立读取放入有界并发组；同时给每个节点设置超时，明确全部成功、部分成功和超时后的处理规则。并行节点不能无限启动，需要控制并发和连接池，避免把单请求时延优化成下游雪崩。

对于 `approve`、`reject`、状态变更和通知等副作用动作，还要结合状态机、幂等键和补偿设计。最终用 Trace 和压测数据证明关键路径缩短，而不是仅凭代码结构判断“这样最快”。

D. Agent、AI 编程与知识治理

23. 实习期间是否接触过 Agent 和 AI 编程？分别用在什么场景

建议时长：75~90 秒。

我接触的是两类不同场景。第一类是业务 Agent：风控客诉链路会消费其他团队提供的结构化 Agent 结果，并把它作为规则判断的一项输入。我负责结果接入、Schema 校验、超时和异常兜底，不负责模型训练，也不会让 Agent 单独绕过确定性规则直接执行高风险动作。

第二类是 AI 编程：我会用它辅助阅读 PRD 和历史文档、梳理调用关系、生成候选技术方案、分模块实现、补测试用例和分析 QA Bug。但设计、代码、权限边界和上线结果仍需人工评审及实际测试。

原文中的内部工具和模型名称转写不可靠。正式面试前应核对真实名称；核对不了就只讲使用流程，不说具体产品名和版本。

24. 从拿到 PRD 到代码上线，你会怎样与 Codex 等 AI 编程工具协作

建议时长：90 秒。

我会把 AI 放进现有研发流程，而不是让它替代流程。第一步先读 PRD，和产品、运营及导师确认业务目标、范围、异常路径、成功指标和合规边界；第二步向 AI 提供经过授权的最小上下文，让它先检索现有模块、接口和历史方案，再按模块给出候选设计和风险清单。

方案经人工评审后，再把任务拆成小批次实现，每批要求引用现有代码路径、优先复用、补单元测试和边界用例。我会人工 Code Review，重点检查业务语义、并发、幂等、权限、错误处理和是否虚构接口。之后运行静态检查、单测、集成测试和故障场景，提交 QA；Bug 可以让 AI 辅助定位，但修复仍要复现和验证。

最后按团队流程灰度、监控和回滚。未授权的公司代码、用户数据和风控规则不能发给外部模型，AI 生成内容也不能绕过评审直接上线。

25. 需求澄清阶段，你怎样判断应该向产品、运营或研发同学询问哪些问题

建议时长：75 秒。

我不会只凭临场感觉，而是沿着需求链路提问。先确认为什么做、目标用户是谁、当前流程和痛点是什么、成功指标如何定义；再确认范围和边界，包括哪些案件包含或排除、异常与人工兜底、权限和合规要求。

然后确认系统事实：输入字段从哪里来、哪个系统是权威源、上下游 Owner 和接口 SLA、重复请求和超时如何处理、历史数据是否迁移。最后确认交付：验收用例、灰度范围、监控、回滚和上线时间。

业务目的、审核口径问产品和运营；存量架构、接口约束、部署与故障经验问导师或对应研发。问题应基于 PRD、代码和现有文档中的具体缺口，而不是让对方重新讲一遍所有背景。

26. 团队怎样避免实习生或新人反复询问同一批项目背景问题

建议时长：60~75 秒。

核心是把口头知识变成有 Owner、会更新、能检索的团队资产。可以建立统一 Onboarding 入口，包含业务术语、系统地图、关键链路、服务 Owner、开发环境、发布流程、常见故障和首周任务；重要架构取舍用 ADR 记录，接口和运行手册尽量从代码或配置自动生成。

新人先按清单完成阅读和一个小任务，遇到新问题时先检索，再把最终结论补回对应文档。文档要标注 Owner、适用版本和更新时间，并纳入代码评审或发布检查，否则很快会过期。

这样不是完全禁止提问，而是让提问建立在已有上下文上，把导师时间留给真正需要经验判断的问题。

27. 大量项目文档应该怎样组织，才能让新人快速建立全局认识，而不是逐篇阅读

建议时长：75 秒。

我会采用分层和渐进式披露。第一层是一页式入口，只回答团队做什么、核心业务对象、完整主链路、服务地图和 Owner；第二层按业务域或服务提供架构图、关键数据、接口、依赖和常见场景；第三层才是具体技术方案、配置说明、故障复盘和代码细节。

每篇文档应有摘要、适用范围、版本、更新时间、上游/下游链接和关键词，重复文档要合并或标记废弃。新人不需要先读完全部文章，而是先通过主链路建立坐标，再根据当前任务进入对应服务和场景。

AI 可以帮助生成摘要和导航，但关键流程、Owner 和版本必须由团队维护，不能让模型把过期文档总结成新的“事实”。

28. 面对十几个微服务和大量文档，怎样让上下文有限的 AI 快速、低成本地找到相关信息

建议时长：90 秒。

不能把全部文档一次性塞进上下文。我会先建立服务目录和业务知识索引，给文档附加业务域、服务名、文档类型、版本、Owner、权限和更新时间等元数据；文档按标题和语义块切分，保留父标题及来源链接。

查询时先识别业务域和目标服务，再做关键词与向量混合检索，使用权限过滤、Top-K 和重排，只把少量高相关片段送给模型。如果信息不足，再沿服务依赖图或文档链接进行第二轮检索，而不是无边界扫描。回答必须返回来源、版本和不确定项，方便人工回到原文核对。

效果不能只凭“看起来回答对”。要准备真实问题集，评估 Top-K 召回率、引用准确率、答案正确率、延迟和 Token 成本，并分析未命中原因。流程图和总览文档很重要，但它们是路由入口，不是完整知识库。

29. AI 写代码时喜欢新造能力、不复用系统已有实现，应该怎样引导和约束

建议时长：75 秒。

首先让“先检索、后设计、优先复用”成为仓库级规则，并维护可发现的能力目录：组件名称、用途、接口、示例、Owner 和适用边界。给 AI 的任务不能只写“实现功能”，还要要求它先列出检索过的路径和候选复用点，解释为什么复用或为什么确实需要新增。

实现阶段提供相邻模块、调用示例、测试和允许使用的依赖，限制它随意引入库或虚构接口。Code Review 时检查重复实现、接口兼容性、错误处理和依赖方向；可以配合静态检查、依赖规则和重复代码检测，但这些工具不能替代业务评审。

如果系统没有统一目录，只在一份很长的说明文件里写一句“请复用”，规则很容易在长上下文中失去作用。真正有效的是让复用能力本身可检索、有示例、可验证。

30. 当规则很多、长指令容易被忽略时，怎样让 AI 在开发的具体阶段知道应复用哪些能力**建议时长：75~90 秒。**

不要让模型每次加载所有规则，而是按研发阶段建立门禁。规划阶段先调用服务目录或代码搜索，输出依赖图和复用候选；实现阶段只加载目标模块的接口规范、示例和局部规则；提交前再执行测试、静态检查和架构约束。每个阶段都有明确输入、输出和验收条件。

通用规则放在短小的仓库入口，领域规则按目录或服务拆分，通过元数据和触发条件按需加载。对于高价值能力，还可以提供固定模板或工具接口，让 AI 直接调用，而不是靠自然语言记住十几种用法。

最后要求 AI 在变更说明中列出复用了哪些组件、为什么没有使用其他候选，并由测试和 Code Review 验证。这样把“记住规则”转化为“流程中必须通过的检查点”。

31. 团队 Skill 越来越多时，怎样做分类、路由、版本和生命周期管理**建议时长：90 秒。**

我会把 Skill 当作受治理的团队资产，而不是个人提示词集合。每个 Skill 至少要有唯一名称、简短描述、触发条件、输入输出、权限、副作用、Owner、版本、适用服务、示例和评测用例；先按通用、业务域、服务和具体任务分层，入口只暴露摘要，命中后再渐进式读取正文和引用资料。

路由可以先按元数据、关键词和目标目录筛出少量候选，再由规则或模型选择；涉及发布、数据修改等副作用的 Skill 必须有权限白名单和人工确认。不能让模型为了选择一个 Skill 先全文阅读整个仓库。

生命周期上要做重复检测、评审、版本固定、变更日志、使用统计和定期清理。效果差或长期无人维护的 Skill 应标记废弃并提供替代项；关键 Skill 要用固定任务集做成功率、误触发率、Token 成本和安全回归测试。这样才能在数量增长后仍保持可发现、可维护和可控。

E. 英文表达**32. 请用英语简单介绍一个实习项目，包括所在团队、项目目标、个人贡献和收获****建议时长：45~60 秒。**

可以直接使用下面这版，句子不复杂，但信息完整：

During my internship at ByteDance, I worked in the Risk Platform team of the international payments department. I helped build an automated review workflow for a subset of merchant appeal cases that

had previously been handled manually. My main responsibilities were integrating structured case data and internal services, implementing configurable rules and action orchestration, and adding fallback handling for uncertain or failed cases. This experience taught me how to translate changing business requirements into reliable backend workflows and how to collaborate with product, operations, and upstream and downstream teams. It also made me more interested in backend engineering for global financial services.

表达时按“团队 -> 项目 -> 个人贡献 -> 收获”四段走，每句说完短暂停顿。不要临时补直属领导姓名、未经核验的指标或复杂从句；**colleague** 表示同事，**department** 表示部门，**appeal** 表示申诉。

F. 候选人反问与面试反馈

33. 为什么本轮没有安排算法题

原现场回答记录与解读。

面试官表示，不同面试官的关注点不同。他本人来自风控方向，因此本轮更希望了解候选人对业务和风控链路的掌握，没有重点安排算法题；他同时强调这只是个人倾向，不代表其他面试官。

正确解读是：本轮不考算法不等于滴滴或该岗位不重视算法，更不能因为 AI 能写算法就停止准备。后续仍应继续复习常见数据结构、算法和现场编码，同时加强业务建模、复杂链路和英文表达。

34. 面试官怎样评价本场表现？后续最需要改进什么

原现场回答记录与改进方案。

面试官认为候选人的面试经验较丰富，也能回答不少问题；主要不足是对风控业务还需要继续深入。介绍项目时不能只说“产品给我需求后，我完成了哪些代码”，还要说明上游为什么产生这个案件、下游执行什么动作、为什么需要这些动作，以及它们对业务和风险结果有什么影响。

下一轮前应准备一张脱敏端到端流程图，按“事件来源 -> 在线决策/处罚 -> 申诉进入运营平台 -> 事实与 Agent 结果 -> 规则判断 -> 下游动作 -> 人工兜底 -> 质量反馈”讲清楚，并为每个节点核对 Owner、事实源、失败路径和个人边界。

35. 滴滴国际化金融团队目前怎样使用 AI 编程，AI 生成代码的参与程度如何

原现场回答记录与边界。

面试官表示，团队当前广泛使用 AI 辅助写代码，并认为在交付节奏下只依赖手写很难高效完成需求。这能说明该面试官所在团队对 AI 编程持积极态度，但这是一次面试中的个人表述，不能外推成滴滴所有团队统一要求，更不能表述为“所有代码都由 AI 独立完成”。

如果继续追问，可以关注更有价值的工程问题：公司如何控制代码和数据外发、怎样选择可用模型、AI 代码是否必须人工评审、单测和质量门禁如何设置、哪些高风险模块限制使用、如何统计采纳率和缺陷率。真正的能力不是生成代码占比，而是能否用 AI 提效同时守住正确性、安全和可维护性。

六、下一轮面试前核验清单

1. 国际支付风控完整业务链路中，各中心的准确名称、Owner、输入输出和本人是否真实接触过。
2. 本人负责的客诉案件类型、结构化字段类别、实际动作类型和人工 fallback 条件。
3. 内部配置平台、表达式引擎、业务服务、AI 编程工具和知识工具的准确名称。

4. 链路真实超时配置、P95/P99、主要慢点、重试策略，以及是否有实际并行优化。
5. 配置校验、审批、灰度、回滚、监控和审计中本人实际参与的部分。
6. Agent 输出的 Schema、超时和异常处理方式，以及它在最终规则中的真实权重和边界。
7. 是否有可核验的自动化覆盖率、抽检一致率、误放/误拒、人工节省或区域上线数据；没有证据就不报数字。
8. 把第 32 题英文介绍录音练习三遍，控制在 60 秒内并保证完整说完。